Back to Python

Iteration and simulation in Python

Recall our first simulation example:

```r
M <- 10 # number of people at the party
hats <- 1:M # numbered hats
nsim <- 10000 # number of simulations
results <- rep(0, nsim) # vector to store the results

for(i in 1:nsim){
  randomized_hats <- sample(hats, M, replace = FALSE)
  results[i] <- sum(randomized_hats == hats) > 0
}

mean(results)
```

```
[1] 0.6324
```

Iteration and simulation in Python

Here is the same code, written in Python

```
1  import numpy as np
2
3  M = 10 # number of people at the party
4  hats = np.arange(M) # numbered hats
5  nsim = 10000 # number of simulations
6  results = np.zeros(nsim) # to store the results
7
8  for i in range(nsim):
9      randomized_hats = np.random.choice(hats, M, replace = False)
10     results[i] = np.sum(randomized_hats == hats) > 0
11
12 np.mean(results)
```

```
np.float64(0.628)
```

What similarities and differences do you notice?

Step 1: representing the hats

```
1 import numpy as np
2
3 M = 10 # number of people at the party
4 hats = np.arange(M) # numbered hats
5
6 hats
```

0, 1, ..., M-1

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
1 hats[0]
```

```
np.int64(0)
```

```
1 hats[1]
```

```
np.int64(1)
```

- `hats` is a 1-dimensional array (similar to a vector in R)
- Python is *0-indexed*: the first entry is `hats[0]`

Step 2: everyone draws a random hat

```
1 import numpy as np
2
3 M = 10 # number of people at the party
4 hats = np.arange(M) # numbered hats
5
6 randomized_hats = np.random.choice(hats, M, replace = False)
7
8 randomized_hats
```

```
array([1, 7, 0, 6, 5, 3, 9, 4, 2, 8])
```

- `np.random.choice` works like R's `sample` function
- Booleans in Python are `True` and `False` (as opposed to `TRUE` and `FALSE`, or `T` and `F`)

Step 3: check who got their original hat

```
1 import numpy as np
2
3 M = 10 # number of people at the party
4 hats = np.arange(M) # numbered hats
5
6 randomized_hats = np.random.choice(hats, M, replace = False)
7 randomized_hats
```

```
array([9, 5, 6, 3, 0, 8, 7, 1, 2, 4])
```

```
1 randomized_hats == hats
```

```
array([False, False, False,  True, False, False, False, False, False,
       False])
```

```
1 np.sum(randomized_hats == hats)
```

```
np.int64(1)
```

- NumPy arrays allow for “vectorized” operations, like in R

Step 4: iteration

```
 1 import numpy as np
 2
 3 M = 10 # number of people at the party
 4 hats = np.arange(M) # numbered hats
 5 nsim = 10000 # number of simulations
 6 results = np.zeros(nsim) # to store the results
 7
 8 for i in range(nsim):
 9     randomized_hats = np.random.choice(hats, M, replace = False)
10     results[i] = np.sum(randomized_hats == hats) > 0
11
12 np.mean(results)
```

like rep(0, nsim) in R

```
np.float64(0.634)
```

range(nsim) is similar to 1:nsim in R

We don't use the curly braces { } in Python

Instead we use whitespace

Class activity

Work on the class activity (course website).